

Post-Deployment Steps

These manual steps must be completed after every deployment (Workbench or SF CLI).

1 — Verify Flow is Active

1. **Setup** → **Flows**
2. Find **Verification Code Login Flow**
3. Status must be **Active** ☒

If it shows as **Draft**: open the flow → click **Activate**.

2 — Assign Flow to Standard Login

1. **Setup** → **Identity** → **Login Flows** → **New**
 2. Flow: **Verification Code Login Flow**
 3. Login Page: **Default Login Page** (or your My Domain login page)
 4. Click **Save**
-

3 — Assign Flow to SSO (Google SAML)

1. **Setup** → **Single Sign-On Settings**
 2. Click **Edit** on your Google SAML configuration
 3. Under **Login Flow**, select **Verification Code Login Flow**
 4. Click **Save**
-

4 — Schedule the Cleanup Job

The cleanup job is **not auto-scheduled**. Run this once in Developer Console:

```
System.schedule(  
    'VerificationCodeCleanup Daily',  
    '0 0 2 * * ?',  
    new VerificationCodeCleanup()  
);
```

Verify it's scheduled: **Setup** → **Scheduled Jobs** → should see **VerificationCodeCleanup Daily**.

The job runs every day at 2 AM and deletes **Verification_Code__c** records older than 24 hours.

5 — Smoke Test

Try these three scenarios:

Happy Path

1. Log out of Salesforce
2. Log back in (or via Google SSO)
3. You should see the verification code screen
4. Check your email — a 6-digit code arrives
5. Enter the code → org access granted ☒

Wrong Code / Block

1. Log out and log back in
2. Enter 999999 (wrong code) three times
3. After the 3rd attempt — the blocked screen appears
4. Click **OK** → you are redirected to login.salesforce.com
5. Try logging back in immediately → blocked screen appears again + logout
6. After 10 minutes → you can log in normally again ☒

Refresh While Blocked

1. After reaching the blocked screen, **refresh the browser (F5)**
2. The flow should immediately show the force-logout screen ☒
3. Click **OK** → redirected to login.salesforce.com

Admin Utilities

Check Who Is Currently Blocked

Run in Developer Console → Execute Anonymous:

```
DateTime tenMinsAgo = DateTime.now().addMinutes(-10);
List<Verification_Code__c> blocked = [
    SELECT Id, User__c, Attempt_Count__c, CreatedDate
    FROM Verification_Code__c
    WHERE Attempt_Count__c >= 3
        AND Is_Used__c = true
        AND CreatedDate >= :tenMinsAgo
    ORDER BY CreatedDate DESC
];
for (Verification_Code__c v : blocked) {
    System.debug('Blocked User: ' + v.User__c + ' at ' + v.CreatedDate);
}
System.debug('Total blocked: ' + blocked.size());
```

Manually Unblock a User

Replace '005XXXXXXXXXXXXXXXXX' with the target User ID:

```
String userId = '005XXXXXXXXXXXXXXXXX';
List<Verification_Code__c> codes = [
    SELECT Id FROM Verification_Code__c
    WHERE User__c = :userId
    AND Attempt_Count__c >= 3
    AND Is_Used__c = true
    ORDER BY CreatedDate DESC
    LIMIT 1
];
if (!codes.isEmpty()) {
    delete codes;
    System.debug('User unblocked: ' + userId);
} else {
    System.debug('User is not currently blocked.');
```

Clear All Verification Records (Testing Only)

```
delete [SELECT Id FROM Verification_Code__c];
System.debug('All verification records cleared.');
```

Abort Cleanup Job (Before Redeploying Apex)

If you need to redeploy the `VerificationCodeCleanup` class:

```
for (CronTrigger ct : [
    SELECT Id FROM CronTrigger
    WHERE CronJobDetail.Name = 'VerificationCodeCleanup Daily'
]) {
    System.abortJob(ct.Id);
}
System.debug('Scheduled job aborted.');
```

After redeployment, re-run the schedule command in step 4.

Changing the Block Duration

The 10-minute block is a single constant in `VerificationCodeChecker.cls`:

```
@TestVisible public static final Integer BLOCK_DURATION_MINUTES = 10;
```

Update this value → redeploy the class → done. No object, flow, or other changes needed.